tell me about longest common sequence problem as an application of dynamic programming , explain this in simple and easy ay to grasp and understand especially for someone not very smart

Let's break down the **Longest Common Subsequence (LCS)** problem and how dynamic prog ramming is used to solve it in a very simple way.

What is the Longest Common Subsequence (LCS) Problem?

Imagine you have two sequences of characters, like two words or sentences, and you want to find the longest sequence of characters that appears in *both* in the same order, though not ne cessarily consecutively. For example, consider these two strings:

- **String 1:** `ABCDGH`
- **String 2:** `AEDFHR`

The LCS here is `ADH` because these letters appear in the same order in both strings. They're n ot next to each other in both words, but the order is maintained.

What is Dynamic Programming (DP)?

Dynamic programming is a method used to solve complex problems by breaking them into s maller, simpler problems. The idea is to solve each small problem once, store the answer, and then reuse that answer when needed later. This avoids doing the same work over and over ag ain.

How Does Dynamic Programming Solve LCS?

1. **Break It Down:** Think about finding the LCS of two strings one character at a time. If the l ast characters of both strings match, you know that this character is part of the LCS, and y ou move to the remaining parts of both strings. If they don't match, you try two things:
   - Ignore the last character of the first string and find the LCS of the remaining part and the second string.
   - Ignore the last character of the second string and find the LCS of the first string and t he rest of the second string.
2. **Use a Table:** We create a table (imagine a grid) where:
   - **Rows** represent characters of the first string (plus an extra row for the empty prefix).

- **Columns** represent characters of the second string (plus an extra column for the empty prefix).

3. **Fill the Table with Answers:**
   - **Base Case:** If one string is empty, the LCS is 0. So, fill the first row and first column with zeros.
   - **Filling Up:** For each cell `(i, j)`, compare the corresponding characters:
     - **If they match:** `dp[i][j] = dp[i-1][j-1] + 1` (You add 1 because you found a common character, plus the best answer for the remaining parts.)
     - **If they don't match:** `dp[i][j] = max(dp[i-1][j], dp[i][j-1])` (You take the best answer from either dropping a character from the first string or the second.)

4. **Build the Final Answer:** The value in the bottom-right cell of the table will be the length of the LCS for the two full strings.

A Simple Example

Let's say we have two very short strings:

- **String 1:** `ABC`
- **String 2:** `AC`

Now, we construct a table where the rows are `""` (empty string), `A`, `B`, `C` and the columns are `""`, `A`, `C`.

1. **Initialize the first row and first column with 0:**
2. **Fill in the table:**
   - For cell (A, A): Characters match, so take 0 (from top-left) + 1 = **1**.
   - For cell (A, C): Characters don't match, so take the max of left or top cell = **1**.
   - For cell (B, A): No match, so max remains = **1**.
   - For cell (B, C): No match, so again, the best is **1**.
   - For cell (C, A): No match, best is **1**.
   - For cell (C, C): Characters match, so from top-left (B, A cell) you get 1 + 1 = **2**.

   The finished table shows the LCS length is **2** (indicating the subsequence `AC`).

In a Nutshell

- **LCS Problem:** Find the longest sequence that appears in two sequences in the same order, even if the characters are not adjacent.

- **Dynamic Programming:** Breaks the problem into small pieces by building a table where each cell represents a subproblem.
- **Step-by-**
  Step: If characters match at a position, add one to the answer for the smaller problem; if not, choose the longer result from two possible ways.

Dynamic programming helps solve LCS by ensuring we do each small calculation only once a nd reuse these answers later, making the solution both efficient and manageable.